

Portfolio Allocation and Markowitz Portfolio Optimization

This chapter deals primarily with Markowitz portfolio optimization. In 1952, Harry Markowitz published a paper that turned out to be one of the most significant academic breakthroughs ever seen in the world of finance. His work changed the way financiers look at their portfolios and formalized methods previously suggested by some practitioners. In this chapter, we are going to discuss portfolio allocation. We are also going to consider various important terms that will be useful when we are allocating assets in a portfolio using Python. Also, we will be implementing Markowitz portfolio optimization with Python. The following are the topics that we will cover in this chapter:

- Sharpe ratio
- Portfolio allocation
- Portfolio optimization
- Markowitz portfolio optimization theory
- Obtaining the efficient frontier in Python – part 1
- Obtaining the efficient frontier in Python – part 2
- Obtaining the efficient frontier in Python – part 3

Technical requirements

In this chapter, we will be using Jupyter Notebook for coding purposes. We will be using both the pandas library and the quandl library. In order to install the quandl library, open the Anaconda Command Prompt and type the following command:

```
conda install -c anaconda quandl
```

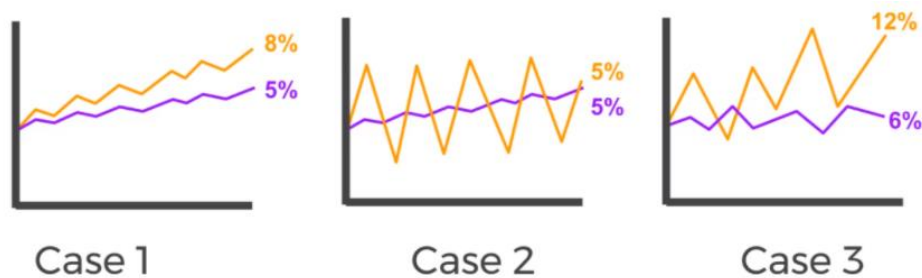
Portfolio allocation and the Sharpe ratio

Usually, when doing algorithmic trading, you're not just dealing with a single stock, you're dealing with a portfolio. A **portfolio** is a set of allocations in a variety of securities. As an example, let's consider a scenario where we have 20% of our portfolio in Apple stocks, 30% in Walmart stocks, and 50% in Google stocks. These percentages are defined as weights. Let's review some of the key statistics related to a stock portfolio:

- **Daily Returns:** The percent returned from one day to the next for each stock
- **Cumulative return:** The amount returned after an entire time period
- **Average daily returns:** The mean of the daily returns
- **Standard daily returns:** The standard deviation of the daily returns

Another critical statistics measure is the Sharpe ratio, which is named after William Sharpe. Let's take a look at what the Sharpe ratio is and why it is necessary.

Imagine we are comparing two portfolios in three different cases. In each case, we are trying to decide which is better:



In the preceding diagram, we have two different portfolios. If we look at **Case 1**, the x axis represents the time period and the y axis represents the total price of the portfolio, with the percentage indicating how much you actually get in return. In the first case, we can see that the top line returns 8%, while the bottom line returns 5%. If we were to decide which portfolio was better in **Case 1**, we would say the upper line, as it has a higher rate of returns.

In **Case 2**, both portfolios return the same value, which is 5%. If you were to choose which portfolio was better in **Case 2**, you might opt for the flatter line, because this one is a lot more stable and has less volatility. The other line from **Case 2** is much more volatile, which means it has a higher risk. Similarly, in **Case 3**, we might say that the stock with a 6% return is more stable, while the stock that has a 12% return is more volatile. We have to decide whether to go for the riskier stock, which will potentially bring higher gains, or to go for the stock with the lower returns but with lower volatility.

The Sharpe ratio allows us to use math to quantify relationships between the mean daily returns and the volatility (or the standard deviation) of the daily returns. It is a measure for calculating a risk-adjusted return and has become the industry standard for this calculation. It was developed by Nobel Laureate William F. Sharpe. The mathematical formula of the Sharpe ratio is as follows:

$$S = \left(\frac{R_p - R_f}{\sigma_p} \right)$$

Here, R_p is the expected portfolio return, R_f is the risk-free return, and σ_p is the standard deviation of the portfolio.

Portfolio allocation and the Sharpe ratio with code

Let's now take a look at a few examples using Python. In this section, we are going to use the stock datasets of a range of tech companies, including Apple, Cisco, IBM, and Amazon. First, we will import the necessary libraries, pandas and quandl, which will help us to retrieve the stock data of a company from an API.

Quandl is a platform for financial, economic, and alternative data that serves investment professionals. Quandl sources data from over 500 publishers. All Quandl's data is accessible through an API. This is possible through packages for multiple programming languages, including R, Python, Matlab, Maple, and Stata.

```
In[4]:
```

```
import pandas as pd
import quandl
```

We will retrieve the past five years' worth of stock data for the four companies using the quandl API, as follows:

```
In[7]:
```

```
start_date = pd.to_datetime('2013-01-01')
end_date = pd.to_datetime('2018-01-01')
```

In[8]:

```
aapl_stock = quandl.get('WIKI/AAPL.11',start_date=start_date,end_date=end_date)
cisco_stock = quandl.get('WIKI/CSCO.11',start_date=start_date,end_date=end_date)
ibm_stock = quandl.get('WIKI/IBM.11',start_date=start_date,end_date=end_date)
amzn_stock = quandl.get('WIKI/AMZN.11',start_date=start_date,end_date=end_date)
```

We have to use the `quandl.get()` function to extract the stock information. After extracting the information, we can save it as a CSV file, as follows:

In[11]:

```
aapl_stock.to_csv('AAPL_CLOSE')
cisco_stock.to_csv('CISCO_CLOSE')
ibm_stock.to_csv('IBM_CLOSE')
amzn_stock.to_csv('AMZN_CLOSE')
```

Please refer the documentation of the quandl library for more information: <https://www.quandl.com/tools/python>.

We also have the CSV files that are provided in the GitHub repository. We can use this data and read the CSV file if Quandl doesn't work due to firewall issues:

```
aapl_stock = pd.read_csv('AAPL_CLOSE',index_col='Date',parse_dates=True)
cisco_stock = pd.read_csv('CISCO_CLOSE',index_col='Date',parse_dates=True)
ibm_stock = pd.read_csv('IBM_CLOSE',index_col='Date',parse_dates=True)
amzn_stock = pd.read_csv('AMZN_CLOSE',index_col='Date',parse_dates=True)
```

First, we will review some important metrics. Let's see whether we can get a value for the cumulative daily returns. We will do this with respect to all the stocks that we have extracted, as follows:

```
In[14]:

for df_stock in (aapl_stock,cisco_stock,ibm_stock,amzn_stock):
    df_stock['Normalize Return'] = df_stock['Adj. Close']/df_stock.iloc[0]['Adj. Close']

In[15]:

aapl_stock.head()
```

The output of the preceding code is as follows:

```
Out[15]:
```

	Adj. Close	Normalize Return
Date		
2013-01-02	71.195748	1.000000
2013-01-03	70.296565	0.987370
2013-01-04	68.338996	0.959875
2013-01-07	67.937002	0.954228
2013-01-08	68.119845	0.956797

The Normalize Return column created gives us the cumulative daily returns. Let's now consider a portfolio that is allocated to the four companies in the following way:

- 30% is allocated to Apple
- 20% is allocated to Google/Alphabet
- 40% in Amazon
- 10% is allocated to IBM

We will multiply the Normalize Return column with these allocations, as follows:

```
In[21]:

for df_stock,allocation in zip([aapl_stock,cisco_stock,ibm_stock,amzn_stock],[.3,.2,.4,.1]):
    df_stock['Allocation'] = df_stock['Normalize Return']*allocation
```

A new Allocation column has been created with respect to the allocation of the portfolio.

The following code is used to display the top five records of the aapl_stock dataframe:

```
In[22]:  
  
aapl_stock.head()
```

The preceding code produces the following output:

```
Out[22]:
```

	Adj. Close	Normalize Return	Allocation
Date			
2013-01-02	71.195748	1.000000	0.300000
2013-01-03	70.296565	0.987370	0.296211
2013-01-04	68.338996	0.959875	0.287962
2013-01-07	67.937002	0.954228	0.286269
2013-01-08	68.119845	0.956797	0.287039

Now, suppose we have invested \$100,000 in this portfolio. We can take this into account as follows:

```
In[23]:  
  
for df_stock in [aapl_stock,cisco_stock,ibm_stock,amzn_stock]:  
    df_stock['Position Values'] = df_stock['Allocation']*100000
```

The following code helps us to see the top five records of the aapl_stock dataframe:

```
In[24]:  
  
aapl_stock.head()
```

The preceding code produces the following output:

```
Out[24]:
```

	Adj. Close	Normalize Return	Allocation	Position Values
Date				
2013-01-02	71.195748	1.000000	0.300000	30000.000000
2013-01-03	70.296565	0.987370	0.296211	29621.108136
2013-01-04	68.338996	0.959875	0.287962	28796.240643
2013-01-07	67.937002	0.954228	0.286269	28626.850992
2013-01-08	68.119845	0.956797	0.287039	28703.895962

As you can see in the preceding output, Position Values specify how much money we allocate to the stocks. You can also see that on January 2, 2013, we initially invested \$30,000. The very next day, this value went down to \$29,621.10 and we continued to drop until the 8th. We can actually see how much money is in our portfolio as the days go by. Let's now create a larger portfolio dataframe that essentially has all of these positioned values for all of our stocks.

We are going to concatenate all the Position Values of all the stocks, as follows:

```
In[25]:  
  
portfolio_val = pd.concat([aapl_stock['Position Values'],cisco_stock['Position Values']
```

The concatenated results are as follows:

```
In[26]:  
  
portfolio_val
```

The output of the preceding code is as follows:

```
Out[10]:
```

	Position Values	Position Values	Position Values	Position Values
Date				
2013-01-02	30000.000000	20000.000000	40000.000000	10000.000000
2013-01-03	29621.108136	20108.161259	39779.984721	10045.470444
2013-01-04	28796.240643	20139.528024	39519.225872	10071.509075
2013-01-07	28626.850992	19951.130777	39346.065699	10433.298356
2013-01-08	28703.895962	19970.501475	39291.061879	10352.493102

From the preceding output, we can see how our \$100,000 is distributed in this portfolio and how this changes over time. We can see how much money we have in our entire portfolio. Currently, all the columns are named Position Values. Let's rename the columns to give us a clear

understanding of which column relates to which stock:

```
In[27]:  
  
portfolio_val.columns = ['AAPL Pos','CISCO Pos','IBM Pos','AMZN Pos']
```

The following code helps us to see the top five records of the portfolio_val dataframe:

```
In[28]:  
  
portfolio_val.head()
```

The output of the preceding code is as follows:

Out[28]:

	AAPL Pos	CISCO Pos	IBM Pos	AMZN Pos
Date				
2013-01-02	30000.000000	20000.000000	40000.000000	10000.000000
2013-01-03	29621.108136	20108.161259	39779.984721	10045.470444
2013-01-04	28796.240643	20139.528024	39519.225872	10071.509075
2013-01-07	28626.850992	19951.130777	39346.065699	10433.298356
2013-01-08	28703.895962	19970.501475	39291.061879	10352.493102

Let's calculate the sum of all the position values on a day-to-day basis and plot this on a graph. This will give us an idea about whether our total money will increase in the future. To do this, we will create a new column, Total Pos, which will have the total sum of all the portfolios on a day-to-day basis.

Let's begin by importing the libraries, as follows:

In[29]:

```
import matplotlib.pyplot as plt
%matplotlib inline
```

The following code is used to create a new column, Total Pos, which will have the total sum of all the portfolios on a day-to-day basis:

In[30]:

```
portfolio_val["Total Pos"] = portfolio_val.sum(axis=1)
```

The following code helps us to see the top five records of the portfolio dataframe:

In[31]:

```
portfolio_val.head()
```

The output of the preceding code is as follows:

Out[31]:

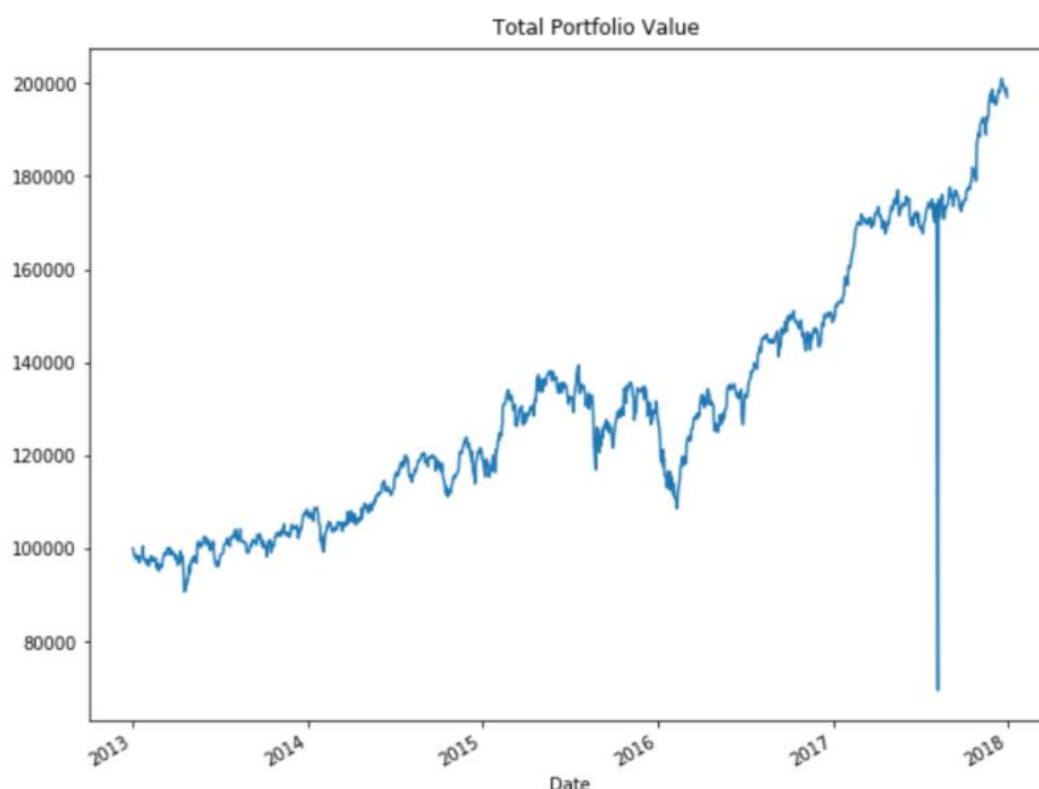
	AAPL Pos	CISCO Pos	IBM Pos	AMZN Pos	Total Pos
Date					
2013-01-02	30000.000000	20000.000000	40000.000000	10000.000000	100000.000000
2013-01-03	29621.108136	20108.161259	39779.984721	10045.470444	99554.724560
2013-01-04	28796.240643	20139.528024	39519.225872	10071.509075	98526.503613
2013-01-07	28626.850992	19951.130777	39346.065699	10433.298356	98357.345824
2013-01-08	28703.895962	19970.501475	39291.061879	10352.493102	98317.952418

Let's plot a figure using Matplotlib for the Total Pos column and see the changes day by day:

In[32]:

```
portfolio_val["Total Pos"].plot(figsize=(10,8))plt.title("Total Portfolio Value")
```

The output is shown in the following screenshot:



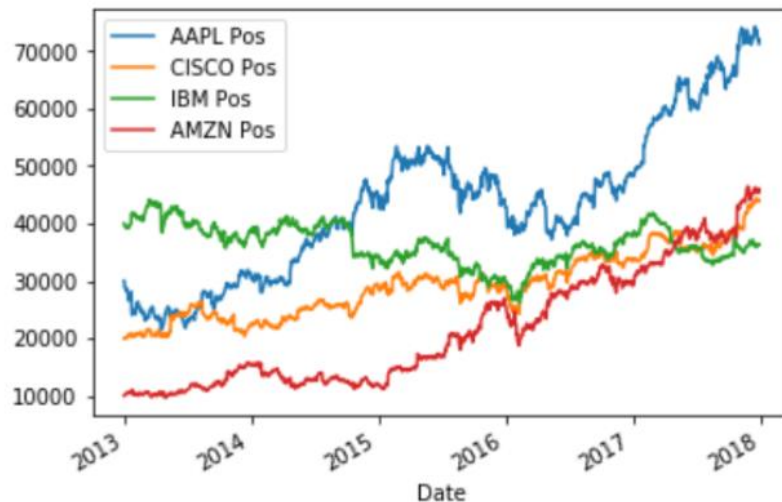
The preceding graph gives us the total sum of all the money invested in the portfolio and the changes that happen on a daily basis. If we need a better plotted diagram to further understand the changes that each company undergoes on a daily basis, we can create one, as follows:

In[33]:

```
portfolio_val.drop("Total Pos",axis=1).plot(kind='line')
```

The output is shown in the following screenshot:

Out[33]: <matplotlib.axes._subplots.AxesSubplot at 0x19b1cba98d0>



Portfolio optimization

In this section, we are going to discuss portfolio optimization. We now have a useful metric, the Sharpe ratio, to evaluate portfolio allocations against each other. Let's consider how to optimize these portfolio holdings. One way to do this is by checking a bunch of random allocations and seeing which one has the best Sharpe ratio. We have powerful computers and we can use NumPy to generate random numbers, so this technique, sometimes known as **Monte Carlo Simulation**, works well. In a Monte Carlo Simulation, we randomly assign a weight to each security in our portfolio. We then calculate its mean daily return and the standard deviation of its daily return. This allows us to calculate the Sharpe ratio for thousands of randomly-selected allocations. We can then plot the allocations on a chart showing returns versus volatility, which is computed by the Sharpe ratio. However, guessing and checking is not very efficient. Instead of this, we can use math to figure out the optimal Sharpe ratio for any given portfolio.

In order to fully understand the optimization algorithm, we first need to understand a concept known as **minimization**. Let's say we have two simple equations:

- $y = x^2$
- $y = (2-x)^2$

Which value of x will minimize the value of y ?

In the first equation, $x=0$ is the value that minimizes the first equation. In the second equation, $x=2$ is the value that minimizes the

equation. This idea of using a minimizer will allow us to build an optimizer. There are mathematical ways of finding this minimum value. Usually, for complex equations such as an optimizer, we can use the `scipy` library to do this math for us.

In the context of our portfolio optimization, we actually want to maximize our Sharpe ratio. Remember that we're trying to figure out which stock allocation will give us the best Sharpe ratio. We want to implement an inverse Sharpe ratio and try to minimize the optimizer, which will end up being the best Sharpe ratio, just in reverse. In other words, we are raising a minimizer in order to calculate the optimal allocation. We will use SciPy's built-in optimization algorithms to calculate the optimal weight allocation for our portfolio and then we are going to optimize it based on the Sharpe ratio.

In the next section, we are going to walk through these steps using Python. We are also going to look at Markowitz portfolio optimization.

Markowitz portfolio optimization

According to Markowitz, investors shouldn't put all their eggs in one basket. Markowitz proved the existence of an efficient set of portfolios that optimize an investor's return according to the amount of risk they are willing to accept. One of the most important concepts he developed was that investments in multiple securities shouldn't be analyzed separately, but should be considered as a portfolio. The financier must understand how different securities in a portfolio interact with each other. Markowitz' model is based on analyzing various risks and returns

and finding the relationship between them using various statistical tools. This analysis is then used to select stocks in a portfolio in an efficient manner, which leads to much more efficient portfolios. Individuals vary widely in their risk tolerance and asset preferences. Their means, expenditures, and investment requirements vary greatly. Because of this, portfolio selection is not a simple choice of any one security or securities, but a subtle selection of the right combination of securities.

In this chapter, we have learned how to measure the relationship between two securities by calculating their covariance. This is precisely what Markowitz suggested. Through a combination of securities with low correlation, investors can optimize their returns without assuming additional risk. Markowitz assumes investors are rational and risk-averse, so they are interested in earning higher returns and prefer avoiding additional risk. This leads to the conclusion that for any level of risk, investors will be interested only in the portfolio with the highest expected return. Usually, when we talk about risk, there are two different types:

- **Systematic risk:** This is the uncertainty that is characteristic of the entire market. Systematic risk is made up of the day-to-day changes in stock prices and is caused by the events that affect all companies.
- **Unsystematic risk:** These are company- or industry-specific risks that can be smoothed out through diversification.

If a portfolio contains two stocks, its risks will be a function of the variances of the two stocks and of the correlation between them. The mathematical formula of the risks of these stocks is as follows:

$$(w_1\sigma_1 + w_2\sigma_2)^2 = w_1^2\sigma_1^2 + 2w_1\sigma_1w_2\sigma_2\rho_{12} + w_2^2\sigma_2^2$$

The equation is described here:

- w_1 : Weight of security 1
- w_2 : Weight of security 2
- σ_1 : Standard deviation of security 1
- σ_2 : Standard deviation of security 2
- ρ_{12} : Correlation between security 1 and 2

Let's now think about a simple example. We're going to look at six different portfolios with two different stocks. The allocations of the two stocks in each portfolio are presented in the following screenshot:

	Share A	Share B
Expected return	7%	9%
Standard deviation	5%	8%
Correlation	30%	

The following quantitative values show the basics weightage along with expected returns in the portfolio:

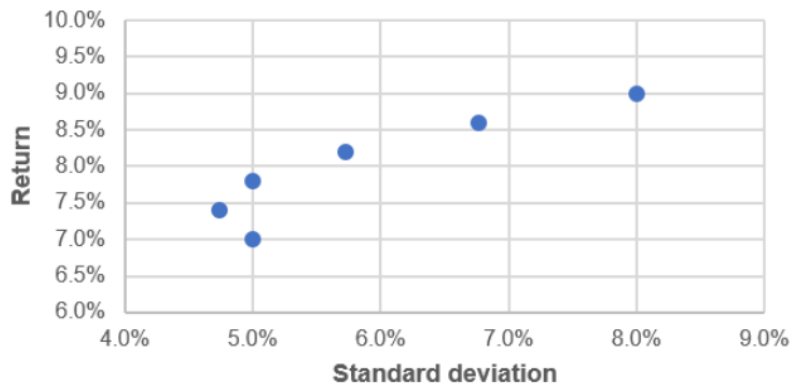
	Weight A	Weight B	Expected Return	Standard Deviation
Portfolio 1	100%	0%	7.00%	5%
Portfolio 2	80%	20%	7.40%	5%
Portfolio 3	60%	40%	7.80%	5%
Portfolio 4	40%	60%	8.20%	6%
Portfolio 5	20%	80%	8.60%	7%
Portfolio 6	0%	100%	9.00%	8%

From these figures, we can see that the first portfolio is made up of 100% of stock A and 0% of stock B. The second portfolio is made up of 80% of stock A and 20% of stock B. The third portfolio is made up of 60% of stock A and 40% of stock B. The fourth portfolio is made up of 40 % of stock A and 60% of stock B. The fifth portfolio has 20 % of stock A and 80 % of stock B, while the sixth portfolio is made up of 100% of stock B.

With respect to these allocations, we can calculate the expected return and the standard deviation.

The standard deviation is the risk of the portfolio. It is plotted on the horizontal axis, while the expected returns are plotted on the vertical axis:

Portfolios 1-6: Markowitz Shape



The preceding diagram specifies the typical shape of a Markowitz efficient frontier. This is precisely what Markowitz suggests. There is a set of efficient portfolios that can provide a higher expected rate of return for the same or an even-lower risk. The starting point of the frontier represents the minimum variance portfolio, the lowest risk an investor could bear. The points under the efficient frontier represent inefficient portfolios. We can use this frontier to find an alternative portfolio with a greater expected return for the same level of standard deviation.

Assumptions of Markowitz's theory

The Markowitz portfolio theory is based on the following assumptions:

- Investors are rational and act in a way to increase a given level of income.
 - Financial specialists have free access to up-to-date data and revise this data based on the returns and risk.
 - The market sectors are effective and assimilate the data rapidly and flawlessly.
 - Financial specialists are risk-averse and attempt to limit risk and
-

amplify return.

- Speculators make choices in light of the expected returns and the standard deviation from the means of these profits.
- Investors would always choose a higher return over a lower return.

Usually, investors select a portfolio that they consider to be efficient. This is the case if no other portfolio or asset offers a return higher than the selected one with the same or a lower level of risk. Usually, investors use portfolio diversification in order to achieve a portfolio that has a low level of risk. This reduces unsystematic risk.

In order to build an efficient set of portfolios, according to Markowitz, we need to consider the following important parameters:

- The expected return
- The variability of the returns, measured by comparing the standard deviation to the mean
- The covariance or variance of one asset's return to the returns of other assets

In general, the higher the expected return, the lower the standard deviation or variance. The lower the correlation, the lower the risk for the investor. Regardless of the risk of the individual securities in isolation, the total risk of the portfolio of all securities may be lower if the covariance of their returns is negative or negligible. In the next section, we will examine more stocks and find the frontier using Python.

Obtaining the efficient frontier in Python – part 1

In this section, we will learn how to calculate the efficient frontier of a group of portfolios composed of two assets: Walmart and Facebook.

To start, import the necessary libraries and read the dataset, as follows:

```
In[3]:

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
companies = ['WMT', 'FB']
df = pd.read_csv('Walmart_FB_2014_2017.csv', index_col='Date')
```

The following code helps us to see the top five records of the dataframe:

```
In[2]:

df.head()
```

The preceding code generates the following output:

```
Out[2]:
```

	WMT	FB
Date		
2013-12-31	71.126411	54.650002
2014-01-02	71.325287	54.709999
2014-01-03	71.090279	54.560001
2014-01-06	70.692558	57.200001
2014-01-07	70.909485	57.919998

We will normalize all the data to the value 100 for plotting the portfolio data on a graph to see how the two assets have been performing throughout the period of interest:

```
In[16]:

df.index=pd.to_datetime(df.index)
```

```
In[17]:

(df / df.iloc[0] * 100).plot(figsize=(11, 6))
```

The output of the preceding code is shown in the following screenshot:

```
Out[17]: <matplotlib.axes._subplots.AxesSubplot at 0x2559dfa4da0>
```



If we want to obtain an efficient frontier composed of these assets or stocks, we need to compute their logarithmic returns. This was discussed in [Chapter 4, Measuring Investment Risks](#). The code for the logarithmic returns of the preceding assets is as follows:

```
In[18]:

log_returns = np.log(df / df.shift(1))
```

The following code helps us to read the top five records of the `log_returns` dataset:

```
In[20]

log_returns.head()
```

The output generated is as follows:

```
Out[20]:
```

	WMT	FB
Date		
2013-12-31	NaN	NaN
2014-01-02	0.002792	0.001097
2014-01-03	-0.003300	-0.002745
2014-01-06	-0.005610	0.047253
2014-01-07	0.003064	0.012509

After that, we need to calculate the mean, covariance, and correlation matrices of the Walmart (WMT) and Facebook (FB) datasets, as follows:

```
In[21]:  
  
#Calculating Mean  
log_returns.mean() * 250
```

The preceding code generates the following output:

```
Out[21]: WMT      0.003551  
         FB       0.287439  
         dtype: float64
```

The covariance is calculated using the following code:

```
In[22]:  
  
# Calculating Covariance  
log_returns.cov() * 250
```

The preceding code generates the following output:

```
Out[22]:
```

	WMT	FB
WMT	0.031544	0.008777
FB	0.008777	0.082786

The correlation is calculated using the following code:

```
In[23]:  
  
# Calculating Correlation  
log_returns.corr()
```

The preceding code generates the following output:

```
Out[23]:
```

	WMT	FB
WMT	1.000000	0.171746
FB	0.171746	1.000000

Note that the mean and the covariance have been multiplied by 250, because there are 250 working days in a year. From the correlation matrix, we see that both the stocks are averagely correlated. The next step is to address portfolio optimization from a coding perspective. First, we will create a variable that will carry the number of assets in our portfolio. This variable is important, as we will be using it in our formulas so they can respond to a change in the number of assets that make up the portfolio. It is equal to the number of elements in the asset list. We can obtain this number with the help of the len function, as follows:

```
In[27]:  
  
num_assets = len(companies)  
  
In[28]  
  
num_assets
```

The output generated is as follows:

```
Out[28]:  
2
```

Here, we are considering two assets: Facebook and Walmart. Remember that the portfolio does not need to be equally weighted. Create a variable called weights. Let this contain as many randomly-generated values as there are assets in your portfolio. Don't forget that these values should be neither smaller than 0 nor equal to or greater than 1.

The next step is to create two random variables using the random function available in the numpy library:

```
In[30]:  
  
arr = np.random.random(2)  
arr
```

The output generated is as follows:

```
Out[30]:  
  
array([0.85216426, 0.73407425])
```

The following snippets helps us to add the initialized values in the array:

```
In[31]:  
  
arr[0]+arr[1]
```

The preceding code generates the following output:

```
Out[31]:  
  
1.5862385086575679
```

By randomly initializing two values, we sometimes find out that the sum of the initialized values is more than one.

To prevent this, we will use the following technique:

```
In[32]:  
  
weights = np.random.random(num_assets)  
weights /= np.sum(weights)  
weights
```

The output generated is as follows:

```
Out[32]:  
  
array([0.50354655, 0.49645345])
```

This gives us the weights of the assets. Using this technique, the sum of the values will always be equal to one. These steps are really important for the next section.

Obtaining the efficient frontier in Python – part 2

Let's push the analysis a few steps further. We will now write the formula for the expected portfolio return. This is given by the sum of the weighted average log returns, which is specified as follows:

```
In[33]:  
  
np.sum(weights * log_returns.mean())* 250
```

The output generated is as follows:

```
Out[33]:  
  
0.1444881608787945
```

The following code will provide us with the expected portfolio variance and the volatility:

- The following code helps us to see the portfolio variance:

```
In[34]:  
  
# Expected Portfolio Variance  
np.dot(weights.T,np.dot(log_returns.cov()* 250,weights))
```

- The output generated is as follows:

```
Out[34]:  
  
# Expected Portfolio Variance  
0.032790341551572906
```

- The following code helps us to see the portfolio volatility:

```
In[35]:  
  
# Expected Portfolio Volatility  
np.sqrt(np.dot(weights.T,np.dot(log_returns.cov()* 250,weights)))
```

- The output generated is as follows:

```
Out[35]:  
  
0.18108103586950486
```

We will need the formulas for the return and the volatility in the simulation of the portfolio's mean-variance combinations. We will now create a graph, where 1,000 mean-variance simulations will be plotted. Pay attention here: we are not considering 1,000 different investments composed of different stocks, we are considering 1,000 combinations of the same two assets, Walmart and Facebook. In other words, we are simulating 1,000 combinations of their weight values. Among these 1,000 combinations, we will probably have one portfolio composed of 1% of Walmart stocks and 99% of Facebook stocks and vice versa. The idea is to compare the two and see which one is more efficient. As mentioned, our goal is to create a graph that visualizes the hypothetical portfolio returns versus the volatilities. We will therefore, need two objects that we can use to store this data. The portfolio returns start as an empty list. We want to fill this with randomly-generated expected returns. We apply a for loop 1,000 times for the portfolio volatilities, as shown in the following code:

In[36]:

```
portfolio_returns=[]
portfolio_volatilities=[]
for x in range(1000):
    weights=np.random.random(num_assets)
    weights/=np.sum(weights)
    portfolio_returns.append(np.sum(weights*log_returns.mean()*250)
    portfolio_volatilities.append(np.sqrt(np.dot(weights.T,np.dot(log_returns.cov(),weights))))

portfolio_returns,portfolio_volatilities
```

The output generated is shown in the following screenshot:

```
Out[36]: ([0.23902430167315575,
0.18944123445616523,
0.17196869286950392,
0.006685968278870664,
0.10990479521109868,
0.16054894843385578,
0.05824036464673471,
0.215181628263505,
0.2780682371520503,
0.18729193595657573,
0.17271604822735367,
0.1636115801655414,
0.12794174155284807,
0.01249290607471217,
0.07250592422643425,
0.20991492017689484,
0.13979037716301046,
0.09211161986228816,
0.13765751454800465,
```

In the loop, we are generating two weights, whose sums equal one. We need two weights, because, as we already mentioned, the portfolios are composed of two assets. As the number of assets increases, the number of weights also increases. We are using the append method, which will add each newly-generated portfolio return value to the list of portfolio returns. This operation will be repeated for each pass of the loop, 1,000 times until the portfolio returns list accumulates 1,000 observations. We repeat the same procedure for the portfolio volatilities. We will use the append method and apply the formula for standard deviation. The output of the preceding code is a list and is hard to manipulate, so we convert it into a NumPy array, as follows:

In[37]:

```
portfolio_returns=np.array(portfolio_returns)
portfolio_volatilities=np.array(portfolio_volatilities)
portfolio_returns,portfolio_volatilities
```

The output generated is shown in the following screenshot:

```
Out[37]: (array([0.2390243 , 0.18944123, 0.17196869, 0.00668597, 0.1099048 ,
                0.16054895, 0.05824036, 0.21518163, 0.27806824, 0.18729194,
                0.17271605, 0.16361158, 0.12794174, 0.01249291, 0.07250592,
                0.20991492, 0.13979038, 0.09211162, 0.13765751, 0.18910214,
                0.08262875, 0.17548627, 0.23482999, 0.04977502, 0.12902758,
                0.07148237, 0.20572807, 0.15286035, 0.26246527, 0.19205318,
                0.15882209, 0.14039463, 0.06006665, 0.20189459, 0.12109478,
                0.07986211, 0.28593727, 0.15261084, 0.20015148, 0.18161396,
                0.10940569, 0.25770711, 0.02861408, 0.21241679, 0.18253775,
                0.19472238, 0.12674706, 0.16290408, 0.19506096, 0.12189415,
                0.17456375, 0.24339411, 0.20517286, 0.20562258, 0.1807871 ,
                0.15753684, 0.04511083, 0.14570486, 0.12292448, 0.13974987,
                0.00435762, 0.20417156, 0.17877689, 0.25769413, 0.04446349,
                0.13165945, 0.06924839, 0.03088512, 0.28478665, 0.12669177,
                0.2450802 , 0.07410808, 0.13628399, 0.11371631, 0.16358597,
                0.17437709, 0.07099607, 0.05968673, 0.27346702, 0.04829892,
                0.13314473, 0.04618645, 0.15861978, 0.1500662 , 0.1096527 ,
                0.05046059, 0.1190782 , 0.09149969, 0.20351993, 0.13591701,
                0.26443873, 0.18947447, 0.18606405, 0.14420315, 0.13771723,
                0.11729746, 0.17779084, 0.13504813, 0.16130019, 0.19122557,
                0.15882209, 0.14039463, 0.06006665, 0.20189459, 0.12109478,
                0.07986211, 0.28593727, 0.15261084, 0.20015148, 0.18161396,
                0.10940569, 0.25770711, 0.02861408, 0.21241679, 0.18253775,
                0.19472238, 0.12674706, 0.16290408, 0.19506096, 0.12189415,
                0.17456375, 0.24339411, 0.20517286, 0.20562258, 0.1807871 ,
                0.15753684, 0.04511083, 0.14570486, 0.12292448, 0.13974987,
                0.00435762, 0.20417156, 0.17877689, 0.25769413, 0.04446349,
                0.13165945, 0.06924839, 0.03088512, 0.28478665, 0.12669177,
                0.2450802 , 0.07410808, 0.13628399, 0.11371631, 0.16358597,
                0.17437709, 0.07099607, 0.05968673, 0.27346702, 0.04829892,
                0.13314473, 0.04618645, 0.15861978, 0.1500662 , 0.1096527 ,
                0.05046059, 0.1190782 , 0.09149969, 0.20351993, 0.13591701,
                0.26443873, 0.18947447, 0.18606405, 0.14420315, 0.13771723,
                0.11729746, 0.17779084, 0.13504813, 0.16130019, 0.19122557,
```

After converting this into an array, we will be able to plot the efficient frontier easily. This will be covered in the next section.

Obtaining the efficient frontier in Python – part 3

We just did most of the important coding. We will now implement the output of the Frontier. First, we will create a dataframe object that contains two columns: one for returns and another one for the respective volatilities. We will call this object portfolios. Its data will be composed of a dictionary with the Return and Volatility keys. These

keys will be automatically assigned column names. The data contained in the portfolio returns and portfolio volatilities arrays will constitute the two key-value pairs and will fill the two columns:

```
In[38]:  
  
portfolios=pd.DataFrame({'Return': portfolio_returns,'Volatility':portfolio_volatilities
```

Let's check the head and the tail of the portfolios dataframe:

```
In[40]:  
  
portfolios.head()
```

The preceding code generates the following output:

Out[40]:

	Return	Volatility
0	0.239024	0.015538
1	0.189441	0.013149
2	0.171969	0.012424
3	0.006686	0.011145
4	0.109905	0.010596

The last five records of the portfolios can be seen as follows:

```
In[41]:  
  
portfolios.tail()
```

The output generated is as follows:

Out[41]:

	Return	Volatility
995	0.273539	0.017409
996	0.102647	0.010477
997	0.175654	0.012571
998	0.014974	0.010930
999	0.062073	0.010251

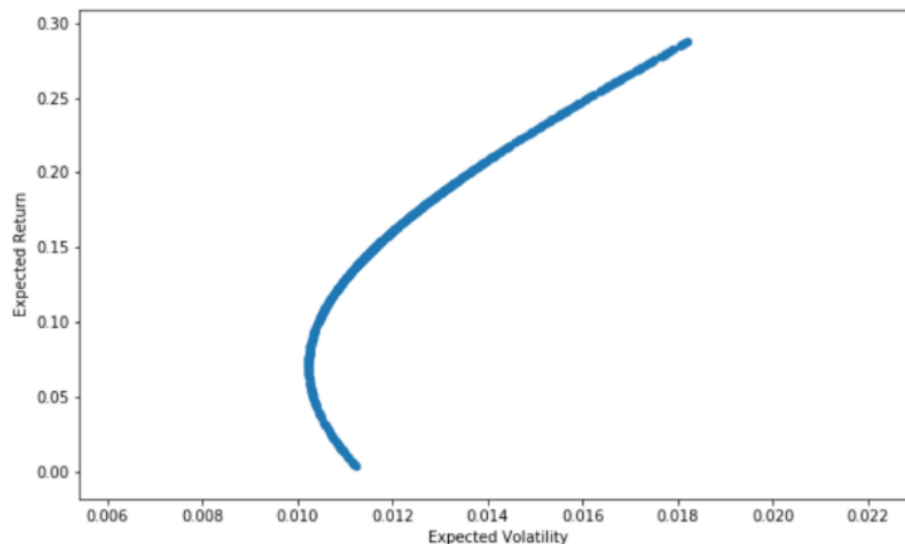
We now need to plot data on a graph with an x axis that corresponds to the Volatility column and a y axis that corresponds to the Return column. An important specification is what kind of graph we want to insert. In our case, we will need a scatter plot. We will also adjust the figure size and provide some labels for the x axis and the y axis:

In[42]:

```
portfolios.plot(x='Volatility',y='Return',kind='scatter',figsize=(10,6))  
plt.xlabel('Expected Volatility')  
plt.ylabel('Expected Return')
```

The output is shown in the following screenshot:

Out[42]: Text(0, 0.5, 'Expected Return')



Summary

In this chapter, we learned about various important concepts, such as portfolios, portfolio allocation, and the Sharpe ratio. We looked at the different things to remember when we are considering investing in portfolios or selecting different assets or stocks in a portfolio. We also learned about the types of risks that we have when we select the different stocks or assets in a portfolio. We looked at the changes in the amount of money invested in various assets in a portfolio on a day-to-day basis, which helps us to determine whether we are gaining or losing money from our investments.

In the preceding output, we see that the curve resembles the Markowitz frontier curve. Go through the notebook file to solidify your understanding of the tools we have presented in this chapter.

Then we discussed Markowitz portfolio optimization, which, in turn, helps us compare various portfolios with different combinations of stocks and different allocation percentages. We also learned how we can create an efficient frontier graph, which helps us understand which portfolios are less risky.

In the next chapter, we will discuss the capital-asset pricing model, which is a model used to determine the theoretically-appropriate required rate of return of an asset, to make decisions about adding assets to a well-diversified portfolio.